

Escuela Politécnica Nacional

Facultad de Ingeniería de Sistemas

Construcción y Evolución de Software

Versión: 1.0

Grupo: 5

Fecha: Enero 2026

Análisis Comparativo y Evaluación de la Funcionalidad en *BraiLator*

Implementación de Requerimiento: Traducción Básica de Braille a Texto Plano

Resumen

Este informe presenta un análisis comparativo entre la rama estable master y la rama de desarrollo develop del repositorio *Braile_checked_web*, con énfasis en la incorporación de un nuevo requerimiento funcional: la traducción inversa de Braille a texto plano. La funcionalidad implementada permite decodificar Braille Unicode Grado 1 hacia lenguaje natural en español, incluyendo soporte para mayúsculas, números, caracteres acentuados y la letra ñ. El análisis se basa en la inspección del código fuente, la revisión de artefactos documentales y la evaluación funcional, evidenciando una evolución significativa del sistema hacia la bidireccionalidad sin afectar la retrocompatibilidad. Esta extensión mejora sustancialmente la accesibilidad, el valor educativo y la utilidad práctica de la aplicación.

1. Introducción

Las herramientas de traducción Braille digitales suelen enfocarse en la conversión unidireccional desde texto plano hacia representaciones Braille, lo que limita su utilidad en contextos de aprendizaje, verificación y corrección de textos escritos por personas usuarias de Braille. La ausencia de mecanismos de traducción inversa dificulta la validación del contenido y reduce la autonomía de usuarios no videntes y de quienes colaboran en su proceso educativo.

En este contexto, el proyecto *Braile_web* aborda progresivamente estas limitaciones. Mientras que la rama master (última actualización: 25 de noviembre de 2025) soporta únicamente la conversión de texto a Braille Unicode Grado 1, la rama develop (última actualización: 23 de enero de 2026) introduce soporte para la traducción inversa, ampliando el alcance funcional del sistema y alineándolo con principios de accesibilidad universal e inclusión digital.

2. Metodología de Análisis

El presente estudio se desarrolla como un análisis comparativo evolutivo de software, empleando las siguientes técnicas:

- Análisis de diferencias entre ramas mediante inspección de commits y cambios (Git diff).
- Revisión estructural y funcional del código fuente.
- Verificación funcional manual a través de la interfaz web.
- Revisión de la documentación técnica generada con Sphinx.

Los criterios de evaluación considerados incluyen: funcionalidad, accesibilidad, retrocompatibilidad, mantenibilidad y alineación con buenas prácticas de ingeniería de software.

3. Discusión y Evaluación de Alternativas de Implementación

Previo a la implementación de la funcionalidad de traducción Braille → texto plano, el equipo de desarrollo sostuvo sesiones de discusión orientadas a identificar la solución más adecuada desde los puntos de vista técnico, económico y de accesibilidad. Como resultado de estas conversaciones, se evaluaron tres alternativas principales:

3.1. Uso de un teclado Braille personalizado

La primera alternativa consistía en el desarrollo o integración de un teclado Braille físico que pudiera conectarse a una computadora y ser interpretado directamente por la aplicación web. Si bien esta opción ofrecía una experiencia de entrada cercana al uso tradicional del Braille, fue descartada debido a que requería un aditamento externo especializado. Esta dependencia implicaba costos adicionales y dificultades de adquisición para las personas usuarias finales, lo que contradecía los principios de accesibilidad y disponibilidad amplia que guían el proyecto.

3.2. Transcripción por voz con conversión posterior a Braille

La segunda alternativa evaluada proponía el uso de reconocimiento de voz para transcribir audio a texto, y posteriormente convertir dicho texto a Braille. Aunque técnicamente viable, esta solución no cumplía con el objetivo central del requerimiento, ya que no permitía a la persona usuaria escribir directamente en Braille. En consecuencia, se determinó que esta aproximación desviaba el enfoque del aprendizaje y uso activo del sistema Braille.

3.3. Entrada directa de Braille mediante interfaz web (alternativa seleccionada)

A partir de las limitaciones identificadas en las propuestas anteriores, se definió una tercera alternativa basada en la entrada directa de patrones Braille a través de una interfaz web interactiva, complementada con soporte para caracteres Unicode Braille. Esta solución resultó ser la más adecuada al no requerir hardware adicional, respetar el proceso de escritura en Braille y permitir la traducción inmediata a texto plano. La alternativa seleccionada equilibra accesibilidad, viabilidad técnica y alineación con los objetivos educativos del sistema.

4. Análisis Comparativo entre master y develop

Dimensión	Rama master	Rama develop
Dirección de conversión	Texto → Braille	Texto ↔ Braille
Soporte de idioma	Español básico	Español completo (tildes, ñ)
Entrada Braille	No disponible	Casillas interactivas y Unicode
Manejo de contexto	No aplica	Indicadores numéricos y mayúsculas
Accesibilidad funcional	Limitada	Ampliada

El diferencial entre ramas evidencia seis commits relevantes, con seis archivos modificados, aproximadamente +1.737 líneas añadidas y -384 eliminadas. Los cambios se concentran principalmente en la lógica de conversión, la interfaz de usuario y la documentación.

5. Descripción de la Implementación

5.1. Lógica de Decodificación Braille → Texto

En el archivo `app.py` se introduce el diccionario inverso `BRaille_TO_TEXT`, construido a partir del mapeo original `BRaille_MAP`. Sobre esta base se implementa la función `braille_a_texto`, la cual realiza una decodificación contextual del Braille Unicode.

La función opera bajo un modelo de estados implícito que permite distinguir entre:

- Estado normal (letras minúsculas).
- Estado de mayúsculas, activado por el indicador `⠠`.
- Estado numérico, activado por el indicador `⠼`.

Este enfoque resulta fundamental para evitar ambigüedades inherentes al sistema Braille, donde un mismo patrón puede representar letras o números dependiendo del contexto.

5.2. Interfaz de Usuario

El archivo `templates/index.html` incorpora una interfaz de entrada Braille basada en casillas interactivas, permitiendo a la persona usuaria introducir combinaciones Braille de forma visual y estructurada. La traducción inversa se muestra en tiempo real, manteniendo responsividad y coherencia visual mediante ajustes en `static/css/style.css`.

6. Integración con la Documentación

La documentación generada con Sphinx, ubicada en `Documentation/build/html`, fue actualizada para reflejar la nueva funcionalidad bidireccional. En particular, los archivos `api.rst` y `uso.rst` describen tanto la API de conversión inversa como el flujo de uso desde la interfaz.

7. Aprendizajes Obtenidos

La implementación evidencia la importancia de los mapeos bidireccionales en sistemas de codificación simbólica y resalta la necesidad de manejar estados contextuales para garantizar traducciones correctas. Desde la perspectiva de ingeniería de software, el trabajo refuerza principios como la separación de responsabilidades, el diseño abierto a extensión y modularidad. Asimismo, demuestra cómo pequeñas extensiones funcionales pueden generar un impacto significativo en términos de accesibilidad y valor social.

8. Conclusión

La incorporación de la traducción Braille → texto plano transforma *Braille_web* en una herramienta verdaderamente bidireccional, ampliando su aplicabilidad educativa y social. La implementación logra este avance sin comprometer la funcionalidad existente, evidenciando una evolución controlada y alineada con buenas prácticas de desarrollo de software accesible.

9. Referencias

1. Repositorio principal del proyecto *Braille_checked_web*.
Disponible en: https://github.com/Construccion-y-Evolucion-Pry/Braille_checked_web
2. Documentación técnica generada con Sphinx - guía de uso y API.
https://github.com/Construccion-y-Evolucion-Pry/Braille_checked_web/tree/develop/Documentation

3. **Unicode Consortium.**

Unicode Standard, Version 15.1 - Braille Patterns (U+2800-U+28FF).

The Unicode Consortium, 2023.

Disponible en: <https://www.unicode.org/charts/PDF/U2800.pdf>

4. **World Wide Web Consortium (W3C).**

Web Content Accessibility Guidelines (WCAG) 2.2.

W3C Recommendation, 2023.

Disponible en: <https://www.w3.org/TR/WCAG22/>